

M**MDI**

v0.1

Modular Data Intelligence

A self-learning document brain that turns any business document — invoices, contracts, claims, certificates, statements — into structured, auditable data.

Upload any document → the brain reads it, extracts every field, learns the pattern, and gets smarter for the next one.

PRODUCT DIRECTION · JUNE 2026

MDI ships as a library kit — not a monolithic platform.

Multiple project teams across the org do document extraction independently — each reinventing the pipeline. **MDI Kit** fixes that: `pip install mdi-kit`, pick a pack, pick your filters, upload a doc, get structured JSON. Runs entirely in the consuming team's environment; Vertex AI primary, Claude fallback; per-project brain (no cross-project leakage in v1). **MDI Studio** (what's already built) remains for pack authoring, correction review, and shared analytics. Full scope in [Part D — MDI Kit v1 initiative](#); design contract in `MDI-KIT-DESIGN.md`.

PART A — FOR BUSINESS STAKEHOLDERS **NON-TECHNICAL**

The problem we solve

Every business receives documents from many vendors in many formats. Each one needs structured data extracted — vendor, account, dates, totals, line items, terms. Today this happens via manual data entry, per-vendor templates that break the moment a layout changes, or rigid OCR rules that don't generalise. **MDI replaces all of that with a learning document brain.**

How it works (the simple version)

When you upload a document, six "organs" of the brain work together — and remember what they learned for next time.

**Eyes**
Looks at the document and identifies what it is — telecom invoice? insurance policy? purchase order?

**Memory (Hippocampus)**
Asks "have we seen this before?" — reuses what it learned, or discovers a new pattern.

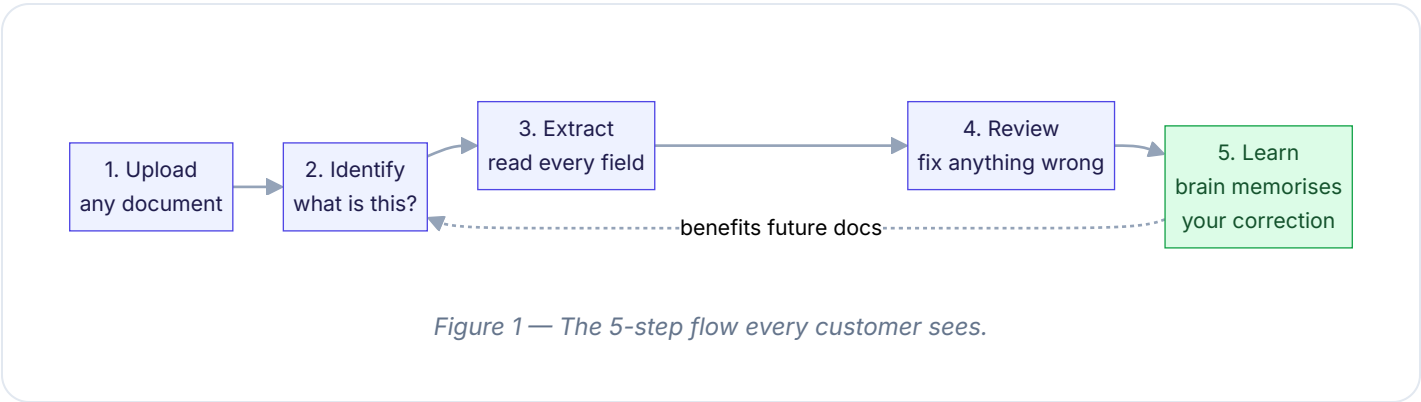
**Hands**
Reads out every field — vendor, account, dates, amounts, line items, addresses, terms.

**Conscience**
Cross-checks the maths and flags anything suspicious (totals that don't add up, expired terms).

**Insight Cortex**
Surfaces what's interesting: recurring vendors, pricing outliers, contracts about to expire.

**Brain memory**
Stores the pattern + every correction, so the next similar document is faster and more accurate.

Customer-facing flow



What makes MDI different

CAPABILITY	TRADITIONAL OCR / TEMPLATES	MDI

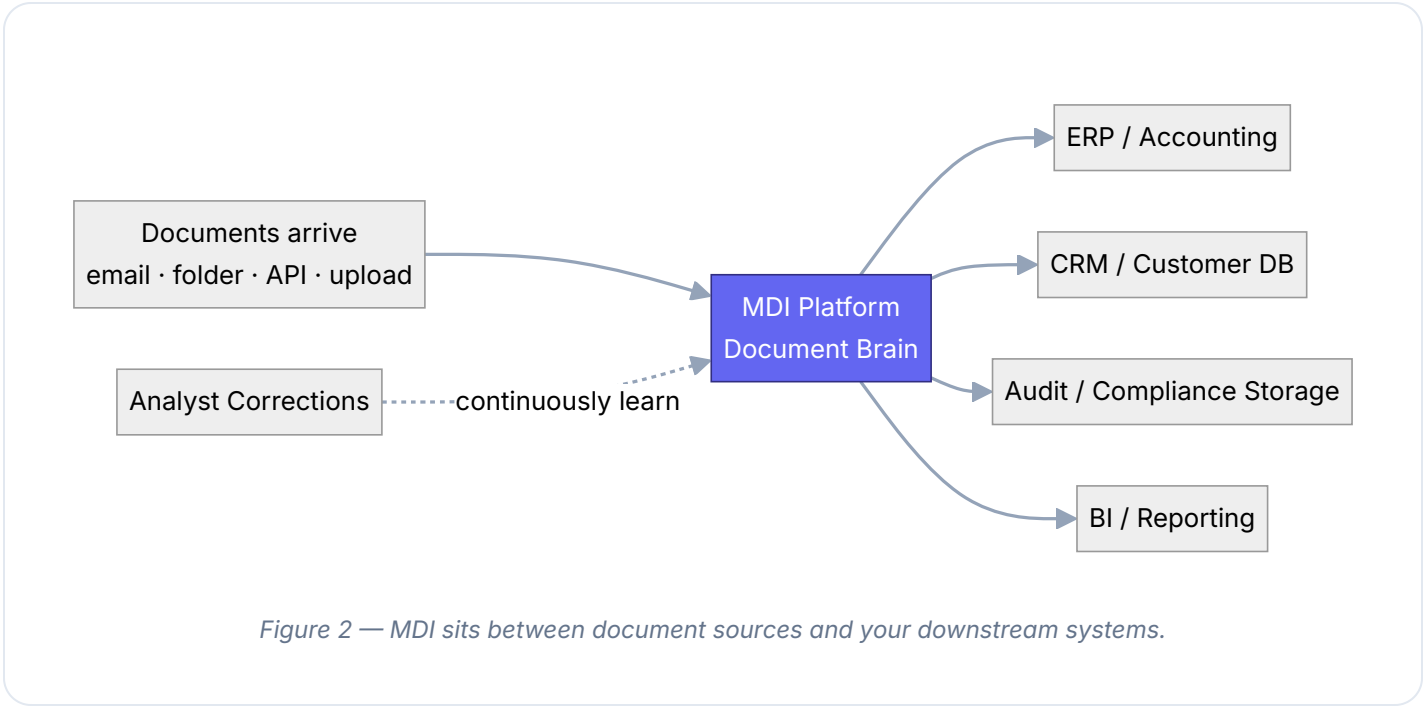
Setup per vendor	Weeks of template authoring	Zero — discovers on first sight
Layout changes	Breaks silently	Adapts automatically
Learning from corrections	Manual rule edits	Corrections embed semantically and bias future extractions
Cross-vertical	One vertical per deployment	One platform, many verticals — packs add insurance / telecom / healthcare
LLM down?	Breaks entirely	Falls back to deterministic regex (~150 ms)
Multi-tenant	Per-customer instances	One platform, customer data isolated by row-level security
Audit trail	Partial / no provenance	Every value carries the exact text that drove it

Two-speed processing

You choose per upload — both produce the same output shape so the rest of the UI doesn't change.

MODE	LATENCY	LLM COST	WHEN TO USE
 Quick	~150 ms / file	\$0	Immediate inspection · demos · LLM rate-limited · batch screening
 Full	~30 s / file	\$0.02–0.20	Production extraction with insights + pattern memory + the full 19-stage pipeline

Where MDI sits in your stack



Stack at a glance

LAYER	TECHNOLOGY	REASON
API	Python 3.11, FastAPI, async SQLAlchemy + asyncpg	Async-native, OpenAPI free, mature in fintech
DB	PostgreSQL 15 + pgvector + HNSW index	One engine for relational, vector ANN, and audit — minimises operational surface
Tenant isolation	Row Level Security + non-superuser app role (mdi_app, NOBYPASSRLS)	Enforced at the database. App bugs cannot leak cross-tenant data.
Embeddings	bge-m3 (1024 dim) via sentence-transformers	Open-weight, on-par with OpenAI ada-002, no vendor lock-in
LLM providers	Gemini 2.5 (Flash + Pro) primary; Claude (Haiku/Sonnet/Opus) fallback	Cost-optimised primary + resilience fallback; same prompt contract via router
VLM (vision)	Gemini 2.5 Pro vision; Claude 3.5 Sonnet vision	Scanned PDFs route via needs_vision flag set by Eyes
Parsing	pdfplumber, Docling	Text-mode + layout-aware modes
Heuristic extractor	Pure Python regex catalogue	Deterministic floor — always-on path even without LLM
Observability	Self-hosted Langfuse + structlog	Per-LLM-call trace UI without sending prompts to a 3rd-party cloud
Cost control	CostTracker (in-memory daily) + SpendLedger (file-backed cumulative)	Hard cap + soft warn, raises before the API call fires
Cache / queue	Redis 7	Celery broker + result backend
Frontend	Next.js 15 (App Router) + React 19 + Material UI 6 + Tanstack Query	Server-rendered hydration, accessible by default, customer-facing aesthetic

Deployment	docker-compose (dev) → containerised services (prod)	Single-command local dev; production-portable
CI	GitHub Actions — ruff + pytest + pgvector service	Lint + 160 offline tests + RLS-verified migration on every push

Layered architecture

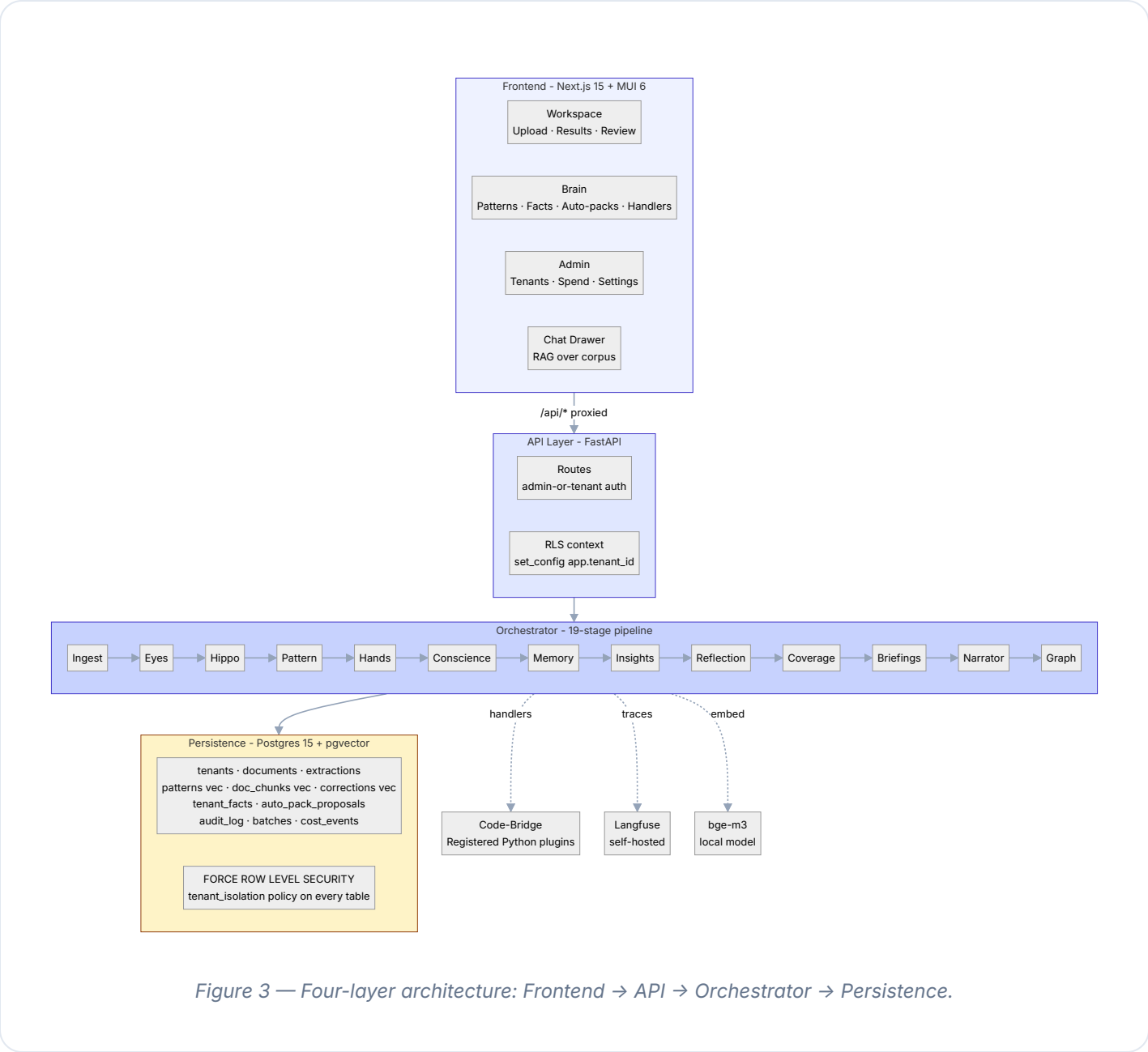
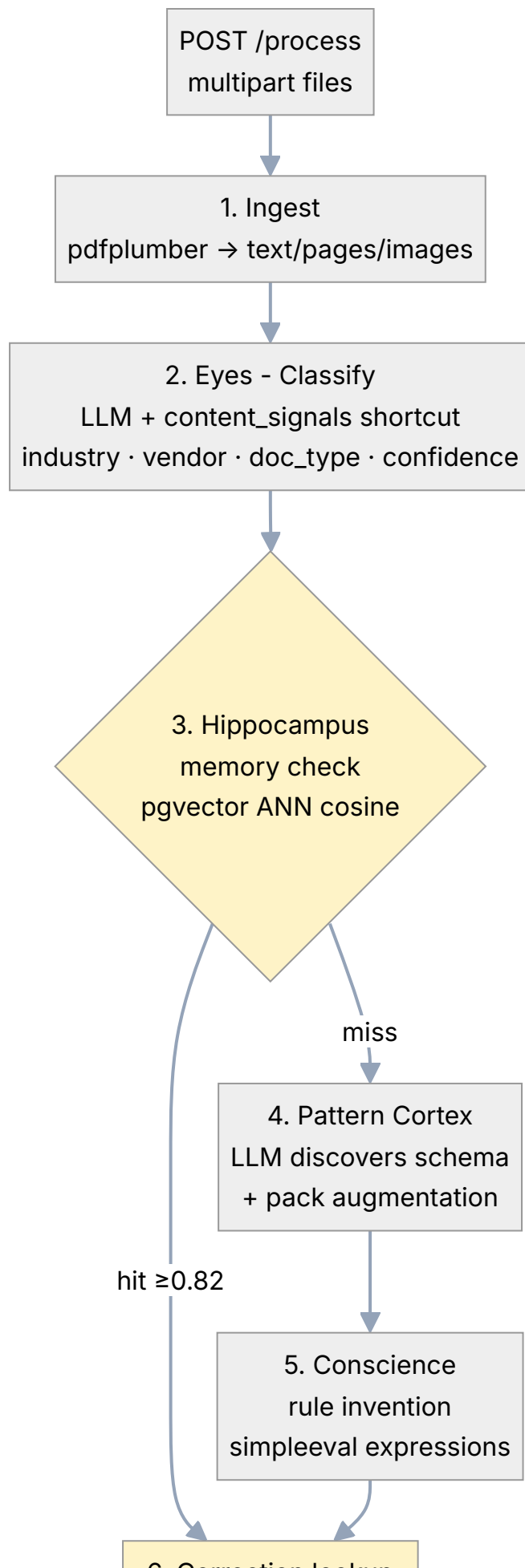
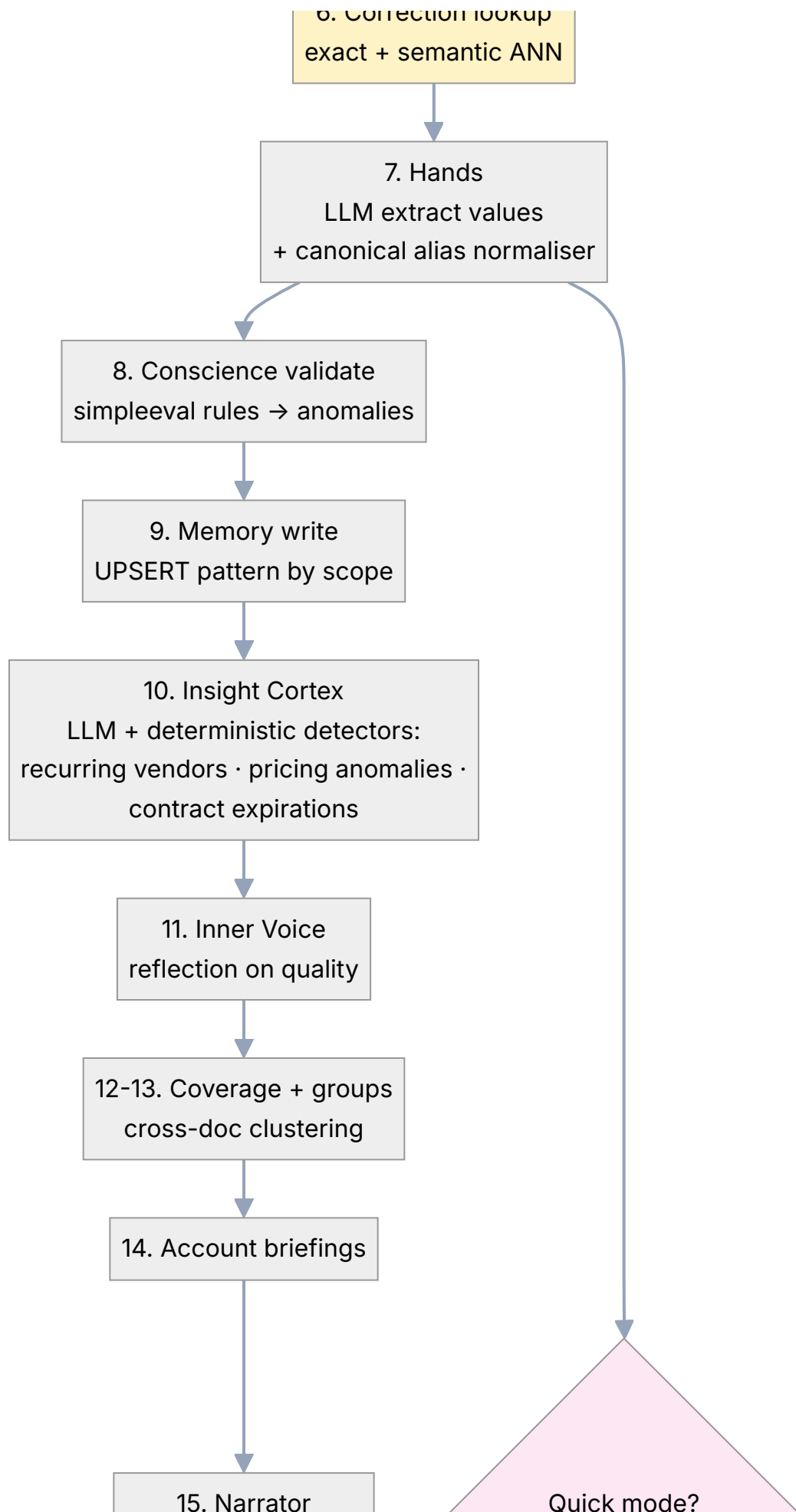


Figure 3 — Four-layer architecture: Frontend → API → Orchestrator → Persistence.

The 19-stage pipeline





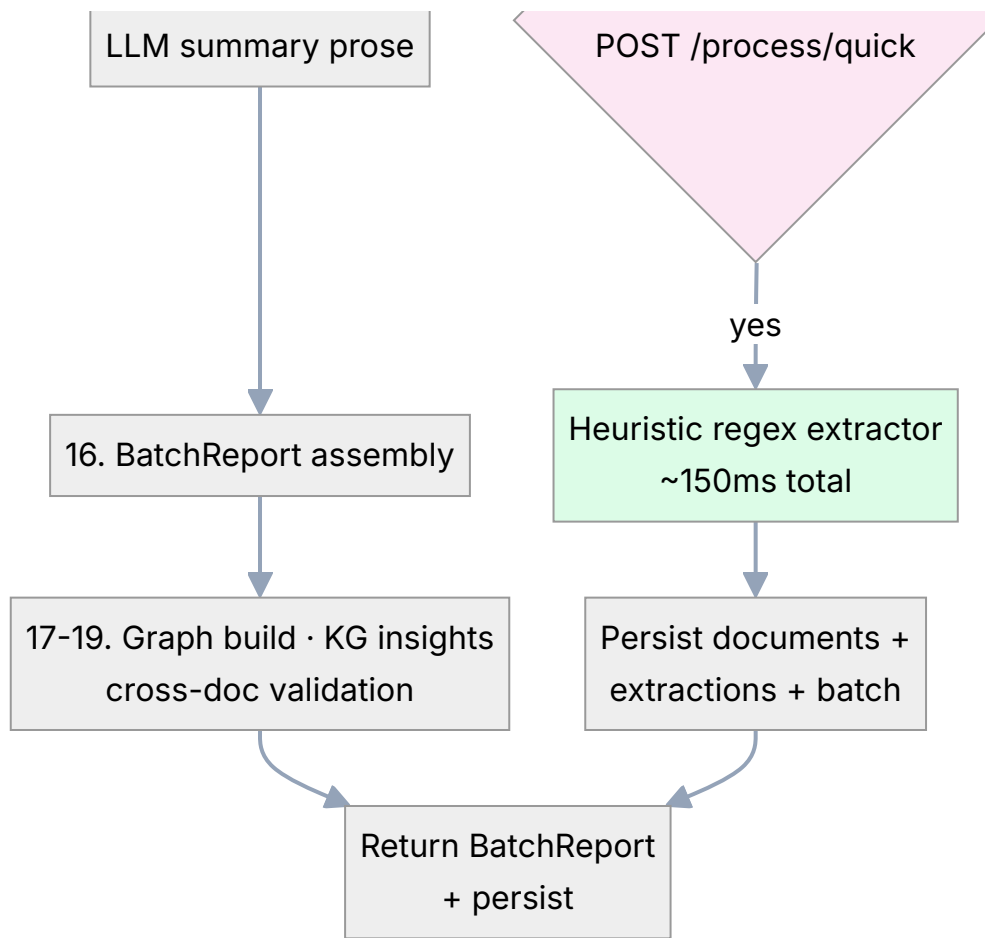


Figure 4 — The full 19-stage pipeline, with Quick-mode short-circuit highlighted.

Two-track extraction strategy

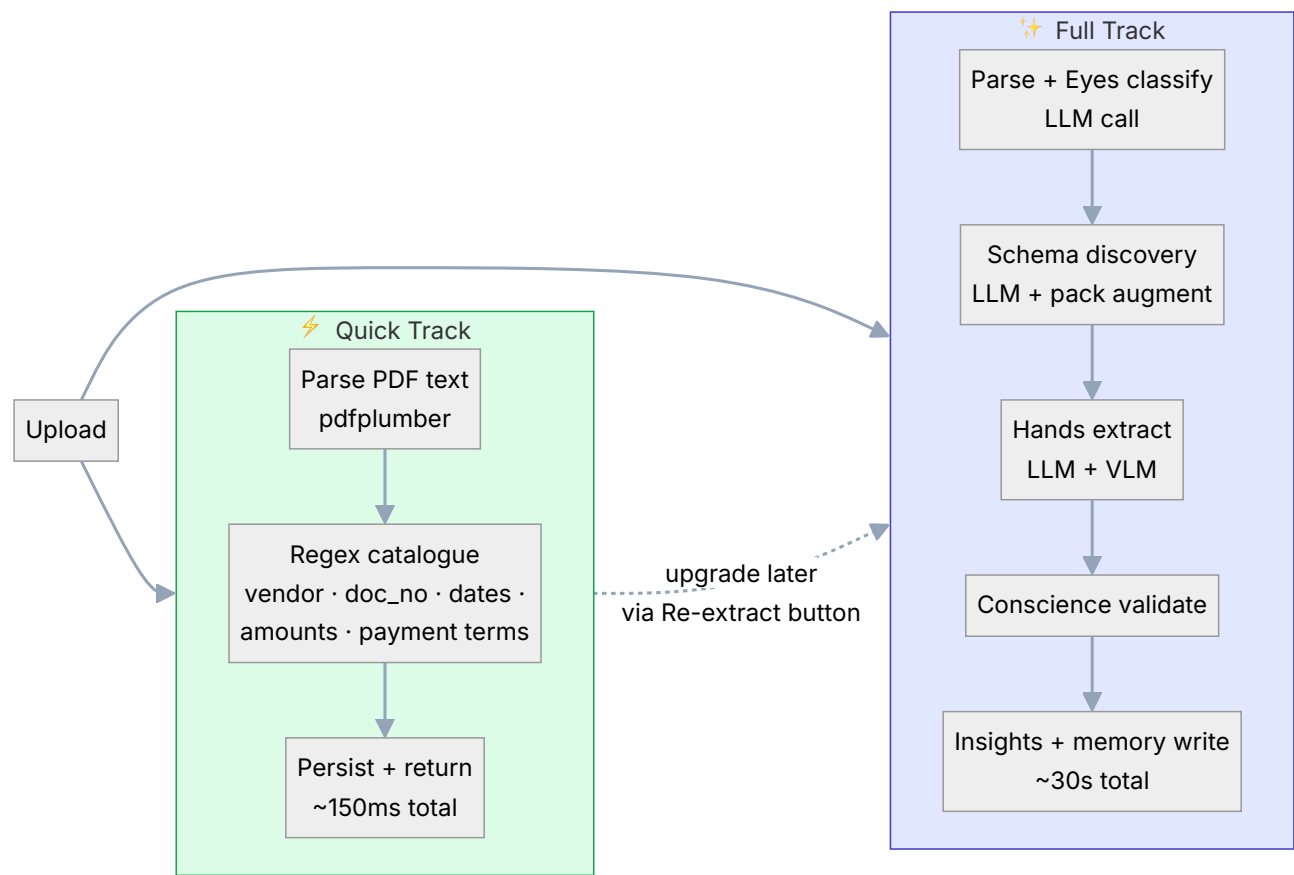


Figure 5 — Two-track extraction: deterministic floor + LLM ceiling.

Multi-tenant security model

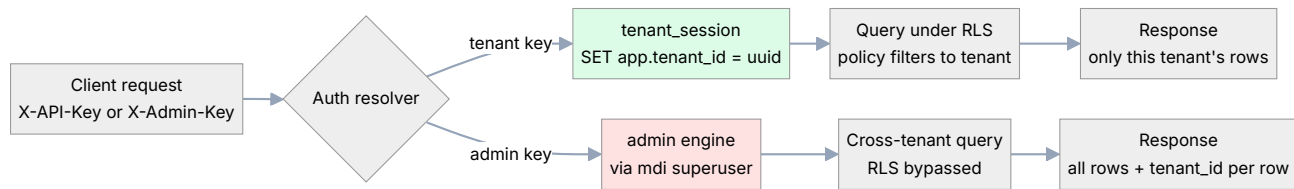


Figure 6 — Dual-engine connection pool: tenant_session (RLS-scoped) vs admin engine (cross-tenant).

The self-training learning loop

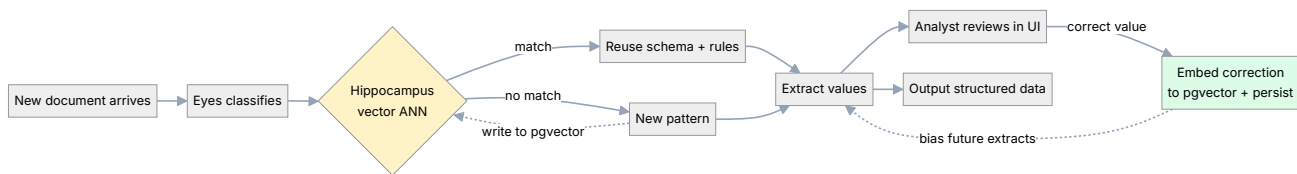


Figure 7 — How the brain learns: every document writes a pattern, every correction biases future extractions.

Brain memory subsystems

SUBSYSTEM	STORES	WHERE	USED FOR
Hippocampus patterns	Schema + rules per (industry, vendor, doc_type)	<code>patterns</code> table, <code>vector(1024)</code>	Skip schema discovery on next similar doc
Tenant facts	Stable per-customer truths (account→vendor, contract→expiry)	<code>tenant_facts</code> table	Cross-batch persistent knowledge
Corrections	Analyst edits with embedded context sentence	<code>corrections</code> table, <code>vector(1024)</code>	Bias future extractions toward analyst-approved values
Auto-pack proposals	Newly-discovered vendors awaiting curation	<code>auto_pack_proposals</code>	One-click pack creation; explicit promotion
Handler registry	Vetted Python callables triggerable by chat / patterns	In-process registry	Code-bridge — LLM dispatches but never executes

Key design decisions + reasoning

DECISION	REASON
Open-vocabulary schema (not fixed templates)	Vendor layouts change constantly; templates need maintenance. Pattern Cortex discovers fields per-document; the canonical-alias layer keeps names stable downstream.
RLS + non-superuser app role	Tenant isolation enforced at the DB. App bugs cannot leak cross-tenant data. <code>FORCE ROW LEVEL SECURITY</code> at every migration.
Two-engine DB pool	App routes use mdi_app (NOBYPASSRLS). Admin cross-tenant routes use mdi (superuser). Separate pools prevent accidental scope creep.
Provider fallback chain	Gemini is cheap but rate-limits. Claude is the resilience layer. Same prompt contract; router walks the chain on retryable errors.
Heuristic extractor as Quick mode	LLM availability is never 100%. Regex floor gives an always-on path. Production-grade docs extract 6 fields in 150 ms with provenance.
pgvector instead of a separate vector DB	One DB to operate + back up + secure. HNSW scales to millions of vectors. Cross-table joins (extractions ↔ patterns) stay relational.
Pack abstraction (YAML, no Python)	Adding a vertical is a config commit, not a code release. Pack augmentation unifies LLM discovery + pack-declared fields.
Per-value provenance (source_text)	Every extracted field carries the substring that drove it. Audit + compliance + analyst trust.
Self-hosted Langfuse	Trace every LLM call without sending prompts / customer data to a 3rd-party cloud.
8px grid + WCAG AA	Accessibility by design. MUI Material 3 tokens default-compliant.

API surface (what the UI calls)

ROUTE	PURPOSE
POST /process	Full 19-stage LLM pipeline
POST /process/quick	Heuristic-only fast lane (~150 ms / file)
GET /batches	Recent batches (admin-or-tenant auth)
GET /report/{id}	Full BatchReport for a batch
GET /admin/documents/{id}/text	Raw parsed text per document
POST /admin/documents/{id}/extract-heuristic	Re-run regex extractor on persisted doc
POST /correction	Analyst correction → pgvector embedding
GET /admin/patterns + /{id}	Memorised pattern explorer
GET /admin/tenant-facts	Per-tenant master data
GET /admin/auto-packs + /{id}/promote	Auto-discovered vendor proposals
GET /admin/handlers + /{id}/run	Code-bridge plugins
POST /chat	RAG over the tenant's corpus
GET /admin/spend	Cumulative LLM spend ledger

Operational characteristics

METRIC	TARGET	TODAY
Quick-mode latency	< 500 ms / file	~150 ms
Full-mode latency (LLM up)	< 60 s / file	~30 s
Backend offline tests	green	160 / 160
Ruff lint	clean	clean
Bundle size (first-load JS)	< 250 kB	~200 kB across all routes
Tenant isolation	RLS-enforced	yes + <code>FORCE ROW LEVEL SECURITY</code>
Provider fallback	automatic	Gemini → Claude via router

PART C — PROPOSED SOLUTIONS & FUTURE ROADMAP

FUTURE

Everything below is a deliberate next step — pieces we've designed for but haven't shipped yet. Sections are written so any one of them can be lifted straight into an implementation ticket. Priority pills indicate suggested build order (`P0` ship next → `P3` nice-to-have).

1. Self-training & continuous learning

The brain already memorises patterns and embeds analyst corrections, but the feedback loop only closes when someone opens a document. The capabilities below let the loop close on its own — the platform improves measurably each week without manual intervention.

CAPABILITY	PRIORITY	WHAT IT DOES
Active learning queue	P0	Any extraction with confidence < 0.7 — or where two providers disagreed — drops into a review queue with the source text already highlighted. One analyst click ⇒ embedding written ⇒ all similar future docs benefit. Today this exists implicitly; surface it as a first-class queue with a weekly KPI ("docs improved this week").
Correction propagation	P0	When an analyst corrects a value, run an ANN search over the existing corpus and silently re-extract the top-K most-similar documents. Today we only bias <i>future</i> extractions — this also fixes <i>past</i> ones retroactively.
Pattern confidence decay	P1	Patterns not re-seen in 90 days lose weight in the matching score. Prevents stale layouts from dominating when a vendor changes their template.
Auto-pack auto-promotion	P1	After an auto-pack proposal scores ≥ 0.85 confidence across N consecutive successful matches, promote it to a first-class pack automatically. Today promotion is manual.
Few-shot from corrections	P1	At extraction time, pull the 3 most-similar prior corrections via pgvector ANN and inject them as ad-hoc few-shot examples in the Hands prompt. Today corrections only bias post-hoc — this fronts them straight into the model's context.
Synthetic doc augmentation	P2	Once a pattern has 5+ corrections, ask Gemini Pro to generate 20 synthetic layout variations + value pairs. Feeds the regex catalogue and the evaluation harness without waiting for real-world drift.
Per-tenant prompt overrides	P2	Each tenant supplies a 200-word "house style" preamble prepended to the Hands prompt. Encodes things like "we always treat 'Net 30' as Net-30 from invoice date, never receipt date."
Tenant-scoped fine-tuning	P3	Once a tenant has 10k+ corrections, fine-tune a small open-weight model (Llama 3 8B) per-tenant. Their docs route through the fine-tune first, fall back to Gemini on low confidence.

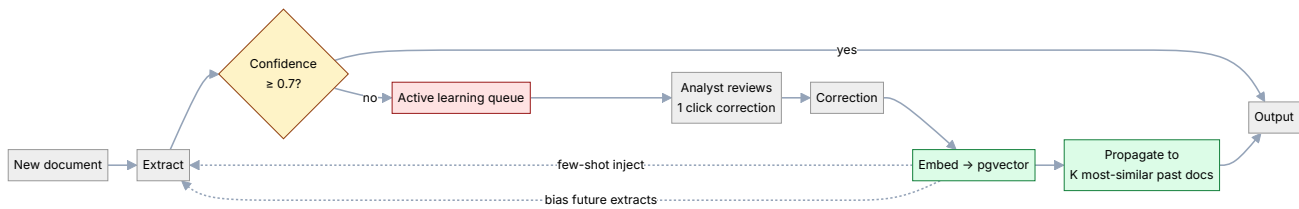


Figure 8 — Closed-loop self-training: low-confidence extractions become training signal automatically; one correction propagates both forward (future docs) and backward (past docs).

2. Domain- & dataset-specific extensibility

Some verticals carry rules that can't be discovered from documents alone — they live in a domain expert's head or in a customer's ERP. We need clean lanes to inject that knowledge without writing Python every time.

Three lanes for adding domain knowledge

LANE	FOR WHOM	MECHANISM
YAML pack	Domain expert (no Python)	Drop a YAML file into <code>packs/<industry>/</code> : schema, rules, validators, enrichment tables. Hot-reloaded by the orchestrator. Already works for insurance + telecom — primary lane going forward.
Code-bridge handler	Engineer (vetted Python)	Register a Python callable in <code>brain/handlers/</code> . The LLM dispatches by name but never executes — the bridge keeps the model out of the call stack. Use for: tax-rate lookups, live FX conversion, ERP API calls, business-logic validators.
Dataset bootstrap	Customer (zero-code)	Upload 5–10 labelled examples through the UI → an LLM-as-judge generates a draft pack YAML + regex catalogue → customer reviews and commits. Best for niche customers ("we want to extract X from our supplier's PO format").

Example — what a customer-uploaded bootstrap sample looks like

```
# POST /admin/bootstrap-pack - 6 labelled samples is usually enough
{
  "vertical": "logistics",
```

```

"vendor_hint": "DHL",
"samples": [
  {
    "file": "sample-1.pdf",
    "labels": {
      "awb_number": "7651234890",
      "origin": "DEL",
      "destination": "FRA",
      "weight_kg": 12.4,
      "chargeable_weight_kg": 14.0,
      "freight_charge_usd": 187.50
    }
  }
  // ... 4-9 more samples ...
]
}

# Brain returns a generated pack scaffold:
# - schema/fields.yaml      ← 6 field definitions inferred
# - rules/validators.yaml   ← weight ≤ chargeable_weight invariant
# - prompts/awb.md         ← air-waybill-specific prompt
# - heuristic regex patches ← AWB number format (10-digit, optional dash)

```

Domain glossary vector store

For domain-specific terminology (medical codes, telecom acronyms, legal Latin), the customer uploads a glossary CSV. Each term is embedded; both the chat layer and the Hands stage ANN-lookup unknown terms to resolve them.

```

# telecom-glossary.csv
term,definition,canonical
MRC,Monthly Recurring Charge,monthly_recurring_charge
NRC,Non-Recurring Charge,non_recurring_charge
USF,Universal Service Fund fee,usf_surcharge
CSR,Customer Service Record,customer_service_record
SIP DID,Session Initiation Protocol Direct Inward Dial,sip_did_number

```

When to hard-code Python (and how to keep it safe)

Most domain rules belong in YAML. But some — live FX rates, tax tables, ERP joins — need real code. The **code-bridge** pattern is the only sanctioned way:

```

# brain/handlers/fx_convert.py
from mdi.brain.bridge import register_handler

```

```
@register_handler(
    name="fx.convert",
    description="Convert an amount between currencies using daily ECB rates.",
    schema={"amount": "float", "from": "str", "to": "str"},
)
def fx_convert(amount: float, from_: str, to: str) -> dict:
    rate = _fetch_ecb_rate(from_, to)          # cached, daily refresh
    return {"value": amount * rate, "rate": rate, "source": "ECB"}

# The LLM emits {"call": "fx.convert", "args": {...}} in its output.
# The bridge validates against the schema, runs the handler, splices
# the result back. The model NEVER executes Python – only describes it.
```

3. Email & inbox integration

Most business documents arrive by email. Today users drag-and-drop into the Workspace — this section makes the platform a first-class participant in the inbox itself.

Supported sources (roadmap)

SOURCE	MECHANISM	NOTES
Gmail / Google Workspace	OAuth2 + Gmail API	Polling, or push via Pub/Sub <code>watch()</code> for near-real-time
Microsoft 365 / Outlook	Graph API + Microsoft auth	Subscription-based push
Generic IMAP	Long-poll IDLE	For self-hosted mail servers
Forwarding alias	Dedicated address (<code>uploads@<tenant>.mdi.app</code>)	Zero-integration option — just forward

What the email parser extracts per inbound message

```
{
  "thread_id": "<CA0j+abc123@gmail.com>",
  "sender": {
    "email": "billing@georgetownpaper.com",
    "name": "Georgetown Paper Stock",
    "domain_trust": "known_vendor"    // known_vendor | new | suspicious
  },
  "subject": "Invoice INV-44128 – May 2026",
```

```
"keywords_matched": ["invoice", "INV-", "month + year"],
"intent_guess": "new_invoice",
"attachments": [
  {
    "name": "INV-44128.pdf",
    "size": 84320,
    "sha256": "e3b0c4...",
    "route": "process_now"
  }
],
"body_extracted": {
  "document_number": "INV-44128",
  "period": "May 2026",
  "amount_due": 12450.00,
  "due_date": "2026-06-30"
},
"linked_documents": ["doc_b8a2..."] // prior invoices from same sender
}
```

Subject-line keyword catalogue (initial set)

The parser classifies intent via a keyword catalogue per vertical. Customers can extend any row.

INTENT	KEYWORDS MATCHED (SUBJECT OR BODY)	ROUTING ACTION
new_invoice	invoice INV- bill statement remittance payment due tax invoice amount due	Auto-process Full mode
purchase_order	P0 # purchase order order confirmation P.O. order ack	Auto-process + match to invoices
contract	agreement MSA SOW renewal addendum amendment NDA	Route to legal-review queue
claim	claim no policy # loss notice adjuster FNOL subrogation	Insurance pack pipeline
receipt / paid	receipt paid payment confirmation thank you for your payment transaction	Reconcile to prior invoice
support / dispute	issue dispute incorrect refund complaint	Skip extraction, queue for human
reminder	overdue past due reminder 2nd notice final notice	Link to existing invoice, raise priority

Auto-routing rules (customer-configurable, hot-reloaded YAML)

```

routing:
  - when:
      sender_domain: "georgetownpaper.com"
      subject_contains: ["invoice", "INV-"]
    then:
      tenant_id: "tenant_acme_corp"
      batch_label: "Georgetown Monthly"
      extraction_mode: "full"
      notify_on_anomaly: "#ops-channel"

  - when:
      sender_domain: "/*.untrusted-domain.tld"
    then:

```

```
action: "quarantine"  
notify: "security@tenant.com"
```

```
- when:  
  subject_matches: "^Claim.*"  
  attachment_count: ">= 1"  
then:  
  pack: "insurance"  
  extraction_mode: "full"  
  assignee: "claims-desk@tenant.com"
```

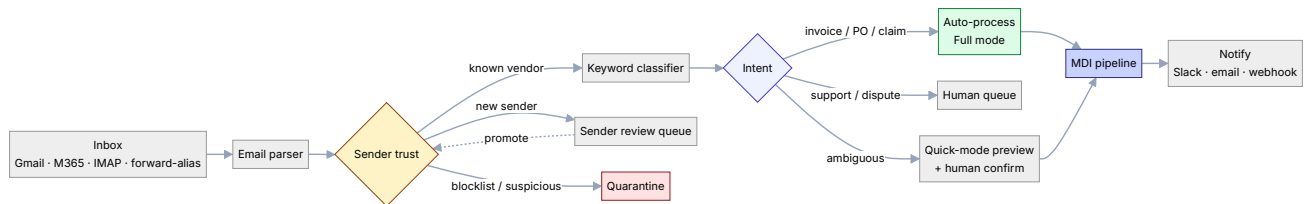


Figure 9 — Inbound email pipeline: sender-trust gate → keyword classifier → intent-routed processing.

4. Expected scenarios & platform behaviour

How the platform should respond to common edge cases — each row is a contract for QA and customer expectations.

SCENARIO	TODAY'S BEHAVIOUR	TARGET BEHAVIOUR
New vendor, first doc	Hippocampus miss → Pattern Cortex discovers schema (~30 s LLM call)	Same, plus surface "first-seen vendor" badge so the analyst pays extra attention
Known vendor, layout changed	Pattern still matches; values may be wrong on new fields	Detect layout drift via embedding distance > threshold ⇒ re-run discovery, version the pattern, keep both
Multi-currency	Currency extracted as string; no conversion	Detect currency code → <code>fx.convert</code> handler ⇒ store both original + base-currency value
Multi-language	Works for EN + basic ES/FR; degrades silently otherwise	Eyes detects language ⇒ language-specific prompt; translation fallback when no native prompt exists
Scanned PDF (image only)	Marked "needs review" — no text extracted	VLM (Gemini 2.5 Pro vision) path: same field schema, OCR-via-LLM, lower confidence cap
Mixed batch (invoices + POs + receipts)	Each doc classified independently — correct, but isolated	Cross-doc reconciliation: receipts → invoices → POs matched automatically inside the batch
High-volume batch (1 k+ docs)	Sync <code>/process</code> times out; must chunk client-side	Async polling endpoint + per-doc status; webhook callback on batch completion
Duplicate document	Re-extracted from scratch each time	SHA256 fingerprint cache ⇒ return prior extraction in < 5 ms
Multi-page table spanning pages	Header repeated ⇒ may extract duplicates	Detect continuation via header similarity; stitch logically into one table
Handwritten annotation on printed form	Often missed	Region-based VLM call on handwriting-detected zones
Two analysts disagree on the	Last-write-wins; silent	Surface as conflict in Brain UI; supervisor resolves; both embeddings

same correction

retained as context

5. Gap closures — concrete tickets

GAP	PRIORITY	APPROACH
Sync <code>/process</code> times out on large batches	P0	Async job model: <code>POST /process</code> returns <code>batch_id</code> immediately; <code>GET /batches/{id}/status</code> returns per-doc state; optional SSE stream for live updates
No webhook notifications	P0	Per-tenant webhook URL + HMAC secret; fire on batch-complete, on each correction, on anomaly threshold
Image-only PDFs marked unreadable	P1	VLM path already exists in code; wire it to the <code>needs_vision</code> flag Eyes already sets
No bulk import from S3 / GCS / SharePoint / Drive	P1	Connector pattern — one credential per source, scheduled poll, idempotent ingest keyed on file path + SHA
Document fingerprint cache	P1	SHA256 the parsed text $\Rightarrow$ cache key $\rightarrow$ extraction JSON for 30 days; saves an LLM call on every repeat
Schema versioning	P2	Each pattern row gets <code>version</code> + <code>supersedes</code> ; old extractions stay queryable under the version they were produced under
Table-aware extraction	P2	Use Docling's structure-aware mode; LLM only on header rows, regex on body rows (already proven 10 $\times$ faster on Georgetown invoices)
Conflict-resolution UI	P2	New Brain tab "Conflicts" lists corrections that disagree on the same field; supervisor picks winner; loser kept as context
Provider routing by document complexity	P3	Cheap doc $\rightarrow$ Gemini Flash; complex doc (long, multi-language, vision) $\rightarrow$ Gemini Pro; fallback chain unchanged
Handwriting-specialised OCR	P3	Detect handwritten zones via VLM bounding-box pass; OCR with a handwriting-specialised model (e.g. TrOCR)

6. Cost & throughput optimisations

LEVER	EXPECTED IMPACT
SHA256 doc-fingerprint cache	~15 % of repeat traffic skips the LLM entirely
Embedding cache (text → vector)	Cuts bge-m3 compute ~40 % on retried batches
Provider routing by complexity	~60 % spend reduction on Flash-eligible docs
Batched LLM calls (multi-doc per prompt)	~25 % latency reduction on small docs
Quick-mode auto-acceptance on high-confidence	~30 % of routine invoices never need a Full-mode call

7. Suggested build order

SPRINT	THEME	TICKETS
Sprint 1 (2 wks)	Async + observability	Async <code>/process</code> · webhook firing · active learning queue UI
Sprint 2 (2 wks)	Email ingestion v1	Gmail OAuth + polling · subject-line classifier · forwarding alias · auto-routing YAML
Sprint 3 (2 wks)	Self-training v1	Correction propagation · few-shot from corrections · auto-pack auto-promotion
Sprint 4 (3 wks)	VLM + handwriting	VLM path wired to <code>needs_vision</code> · scanned-PDF flow · domain glossary uploader
Sprint 5 (2 wks)	Dataset bootstrap	Upload-samples API · pack scaffolder · regex generator · per-tenant prompt overrides
Sprint 6 (2 wks)	Cost + scale	Fingerprint cache · embedding cache · provider routing by complexity

Hold-it bucket — park until a customer asks: tenant-scoped fine-tuning · M365 Graph push subscription · handwriting-specialised OCR · multi-region GDPR sharding · on-prem deployment kit. All are real but speculative until proven demand exists.

This is the current build direction. Everything in Parts A/B/C still holds — but **how** we ship it changes: the extraction brain is packaged as a Python library any project team can embed, not a platform every team logs into. Studio remains for pack authoring and correction review. Full API design contract lives in `MDI-KIT-DESIGN.md` at the repo root.

1. Two products, cleanly separated

PRODUCT	WHAT IT IS	WHO USES IT	SHIPS AS
MDI Kit (new — the library)	The extraction brain, packaged. Import it, pick a pack + filters, get JSON out with confidence + provenance.	Engineers embedding extraction into their own apps.	<code>pip install mdi-kit</code> · CLI · Docker image · REST client (stub)
MDI Studio (what's built today)	The hosted UI + shared learning + pack-authoring surface. Existing FastAPI + Next.js stack.	Pack authors · analysts reviewing extractions · admins.	Existing platform, imports from <code>mdi-core</code> .

Rule of thumb: Studio *produces* packs. Kit *consumes* them. Kit runs in each team's own environment; Studio is where the brain matures. This separation turns MDI from "another platform" into an "extraction primitive."

2. What a consumer's code looks like

```
from mdi_kit import Brain

brain = Brain(
    pack="finance/invoice_v1",
    filters=["vendor", "amount_due", "line_items", "due_date"],
)

result = brain.extract("path/to/invoice.pdf")

print(result.fields)           # {vendor: "GEORGETOWN PAPER STOCK", amount_due: 12450.00, ...}
```

```
print(result.confidence)      # {vendor: 0.94, amount_due: 0.98, ...}
print(result.source_text)     # {vendor: "GEORGETOWN PAPER STOCK\n1234 Mill Rd", ...} ← prov
```

That's the whole embedding cost for a project team. No pipeline setup, no LLM keys in their code, no pgvector. If they want learning + corrections, they graduate to stateful mode with their own Postgres. If they want a UI on top, they wire Studio.

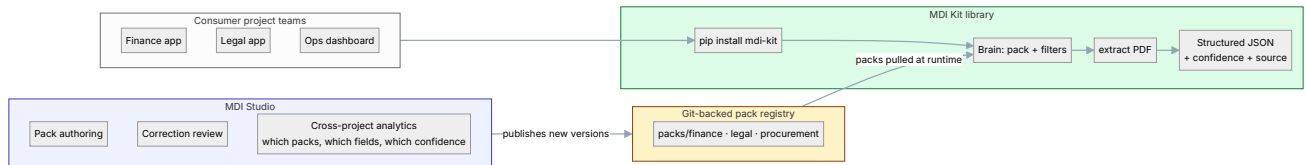


Figure 10 — Kit runs in each consumer's environment; Studio publishes packs to the shared registry; consumers pull packs at runtime.

3. What's in v1

#	DELIVERABLE	WHAT IT IS
1	<code>mdi-kit</code> Python package	Wraps the existing pipeline (Eyes → Hippocampus → Pattern Cortex → Hands → Conscience). PyPI-installable.
2	Pack composability	Teams pick a base pack AND override the filter set. Kit resolves only requested fields.
3	Vertex AI adapter	New adapter in the existing <code>llm_gateway</code> — teams pass GCP project/region/service account. Claude fallback via same gateway.
4	Two run modes	<i>Stateless</i> (extract-and-forget, in-memory) and <i>Stateful</i> (bring-your-own-Postgres with pgvector for the learning loop).
5	CLI	<code>mdi-kit extract --pack invoice_v1 file.pdf</code> for one-off use. <code>mdi-kit packs list/pull/push</code> for pack management.
6	Git-backed pack registry	Packs live in a private org Git repo. Kit pulls/versions them (<code>pack@1.2.3</code>). Zero new infrastructure.
7	3 reference packs	<code>finance/invoice_v1</code> , <code>legal/contract_v1</code> , <code>procurement/po_v1</code> . Real, working, tested.
8	Docs + 3 examples	5-line getting-started, filter composition guide, custom pack authoring guide.
9	Docker image	Optional — for teams that want stateful mode without provisioning their own Postgres. Bundles Postgres + pgvector.

4. Explicitly out of v1 (parked for v2)

DEFERRED ITEM	WHY PARKED
Shared cross-project brain	Requires legal/compliance sign-off per consuming team. V1 keeps brain private per project — no accidental data leakage across teams.
Hosted SaaS API	Library-first was the chosen consumption model. Hosted API stays as a v2 option if a team specifically asks.
JS/TS SDK	Most extraction consumers are backend/data teams. Python covers 90% of demand for v1.
Cross-org pack marketplace	Org-private packs only. Public marketplace is a downstream conversation once the internal registry pattern is proven.
Studio ↔ Kit live sync	Studio publishes packs to registry; Kit pulls on demand. No real-time link needed for v1.

5. The enabling refactor — repository restructure

Today `mdi/src/mdi/` is one monolith. To ship a library cleanly, split into four packages under a monorepo. Studio keeps working — it just imports from `mdi-core` instead of owning it.

```
mdi/
  packages/
    mdi-core/      ← pure extraction pipeline · no HTTP · no UI
    mdi-kit/       ← consumer-facing library · imports mdi-core
    mdi-studio/    ← existing FastAPI + Next.js · imports mdi-core
    mdi-packs/     ← YAML packs · published to git-backed registry
```

This is the **first tangible step** and unblocks everything else. It's also the highest-risk item because the current code has HTTP handlers, DB session management, and pipeline stages tangled together. Untangling them is 1–2 weeks of careful work — but everything downstream is mechanical once it's done.

6. Sequencing (rough 6-week estimate)

WEEK	THEME	MILESTONE
1-2	Core extraction refactor	Extract pipeline from platform into standalone <code>mdi-core</code> . Add Vertex AI provider. Ship <i>stateless</i> mode + minimal CLI. Internal alpha.
3	Pack composability	Filter selection layer. Refactor existing packs to work under the new public interface.
4	Stateful mode + reference packs	Bring-your-own-Postgres mode. Ship <code>invoice_v1</code> , <code>contract_v1</code> , <code>po_v1</code> tested against real samples.
5	Docs + first consumer	Docs site (or README-heavy). First internal team onboarded end-to-end.
6	Iterate + v1.0 tag	Fix whatever the first consumer surfaced. Cut <code>v1.0</code> . Announce internally.

7. Technology stack — what earns its keep

LAYER	CHOICE	WHY
LLM provider	Vertex AI primary + Claude via Bedrock fallback	Vertex gives enterprise auth, org-level quotas, VPC, per-project GCP billing — critical when multiple project teams consume the same kit. Existing gateway abstraction keeps the swap clean.
Observability	Self-hosted Langfuse, one instance per consuming project OR shared with project-tag scoping	Each project team sees only their own traces. Kit auto-tags every LLM call with <code>project_id</code> — no cross-project data leakage.
Parsing	pdfplumber + Docling	Already in the stack. Docling gives layout-aware parsing for tables.
Embeddings	bge-m3 (local via sentence-transformers)	No per-team API key. Ships inside the kit.
Vector store	pgvector in caller's Postgres OR ephemeral in-memory	Kit supports both. Teams that want learning bring a DB; teams that want extract-and-forget don't.
Pack registry	Git-backed (private repo per org), OCI later	Packs are YAML — versioning + PR review + rollback come free from Git. Teams <code>mdi-kit packs pull finance/invoice_v1@2.3</code> .
Filter primitives	Declarative composition layer over packs	Teams compose <code>["vendor", "amount_due"]</code> from any pack that defines them. Kit resolves + runs.

8. What "v1 done" looks like

CRITERION	MEASURABLE TARGET
Consumer onboarding cost	≤ 10 lines of Python from <code>pip install</code> to first structured extraction
First-doc latency (stateless, Quick mode)	< 500 ms per file
First-doc latency (stateless, Full mode via Vertex)	< 45 s per file
Reference packs shipped	3 packs (invoice, contract, PO), each with ≥ 5 sample documents and $\geq 85\%$ field-level accuracy on those samples
Docs completeness	Getting-started · filter composition · custom pack authoring · provider config · troubleshooting — 5 pages minimum
First real consumer	At least one internal team using it against their own document corpus without hand-holding

What we need from stakeholders before build starts: (a) confirm the first consumer project team so we build against their real docs, not synthetic; (b) confirm GCP project structure (each consuming team in their own project, or shared). Neither is blocking — but both meaningfully improve v1 quality.

How to share this: the file is self-contained — Mermaid diagrams render on open via CDN. Open in Chrome, print to PDF, paste into Notion, or upload to Confluence as an HTML attachment.